

一、引言

1.1 为什么需要智能指针？

在C++中没有垃圾回收机制，必须自己释放分配的内存，否则就会造成内存泄露。解决这个问题最有效的方法是使用智能指针（smart pointer）。智能指针是存储指向动态分配（堆）对象指针的类，用于生存期的控制，能够确保在离开指针所在作用域时，自动地销毁动态分配的对象，防止内存泄露。智能指针的核心实现技术是引用计数（请注意unique_ptr无引用计数），每使用它一次，内部引用计数加1，每析构一次内部的引用计数减1，减为0时，删除所指向的堆内存。

1.2 智能指针相比于原生指针有什么优势？

1. 原生指针：原生指针是 C++ 最基本的指针类型，允许程序员直接管理内存。然而，原生指针存在以下问题：
 - 内存泄漏：未释放动态分配的内存
 - 悬挂指针：指针指向已释放或未初始化的内存
 - 双重释放：多次释放同一内存区域
2. 智能指针的优势：
 - 自动销毁：在智能指针生命周期结束时自动释放资源
 - 引用计数：共享智能指针能够跟踪引用数量，确保资源在最后一个引用结束时释放
 - 避免内存泄漏：通过 RAII 机制自动管理资源生命周期
 - 类型安全：提供更严格的类型检查，减少错误

1.3 RAII和智能指针的设计思路

1. RAII是Resource Acquisition Is Initialization的缩写，它是一种管理资源的类的设计思想，本质是

一种利用对象生命周期来管理获取到的动态资源，避免资源泄漏，这里的资源可以是内存、文件指

针、网络连接、互斥锁等等。RAII在获取资源时把资源委托给一个对象，接着控制对资源的访问，

资源在对象的生命周期内始终保持有效，最后在对象析构的时候释放资源，这样保障了资源的正常

释放，避免资源泄漏问题。

2. 智能指针类除了满足RAII的设计思路，还要方便资源的访问，所以智能指针类还会像迭代器类一

样，重载 `operator*`/`operator->`/`operator[]` 等运算符，方便访问资源。

1.4 C++标准库智能指针的使用

1. C++标准库中的智能指针都在 这个头文件下面，我们包含就可以使用了，

智能指针有好几种，除了`weak_ptr`他们都符合RAII和像指针一样访问的行为，原理上而言主要是解

决智能指针拷贝时的思路不同。

2. `auto_ptr` 是C++98时设计出来的智能指针，他的特点是拷贝时把被拷贝对象的资源的管理权转移给

拷贝对象，这是一个非常糟糕的设计，因为他会导致被拷贝对象悬空，访问报错的问题，C++11设计

出新的智能指针后，强烈建议不要使用`auto_ptr`。其他C++11出来之前很多公司也是明令禁止使用

这个智能指针的。

3. `unique_ptr` 是C++11设计出来的智能指针，他的名字翻译出来是唯一指针，他的特点的不支持拷贝，只支持移动。如果不需要拷贝的场景就非常建议使用它。

4. `shared_ptr` 是C++11设计出来的智能指针，他的名字翻译出来是共享指针，他的特点是支持拷贝，

也支持移动。如果需要拷贝的场景就需要使用他了。

5. `weak_ptr` 是C++11设计出来的智能指针，他的名字翻译出来是弱指针，他完全不同于上面的智能指针，他不支持RAII，也就意味着不能用它直接管理资源，`weak_ptr`的产生本质是要解决`shared_ptr`的一个循环引用导致内存泄漏的问题。这个问题我在后面会进行详细解释。
6. 智能指针析构时默认是进行`delete`释放资源，这也就意味着如果不是`new`出来的资源，交给智能指针管理，析构时就会崩溃。智能指针支持在构造时给一个删除器，所谓删除器本质就是一个可调用对象，这个可调用对象中实现你想要的释放资源的方式，当构造智能指针时，给了定制的删除器，在智能指针析构时就会调用删除器去释放资源。

二、`std::unique_ptr`---独占的智能指针

2.1 定义与用法

`std::unique_ptr` 是一种独占所有权的智能指针，任何时刻只能有一个 `unique_ptr` 实例拥有对某个对象的所有权。不能被拷贝，只能被移动。

主要特性：

1. 独占所有权：确保资源在一个所有者下。
2. 轻量级：没有引用计数，开销小。
3. 自动释放：在指针销毁时自动释放资源。

2.2 初始化

`unique_ptr` 提供多种构造函数和赋值操作，以支持不同的使用场景：

- 默认构造函数：创建一个空的 `unique_ptr`
- 指针构造函数：接受一个裸指针，拥有其所有
- 移动构造函数：将一个 `unique_ptr` 的所有权转移到另一个 `unique_ptr`

- 用 `reset()` 方法初始化/重置: `reset()` 是给已创建的空 `unique_ptr` 赋值, 或替换已有 `unique_ptr` 指向的资源

1. 默认构造函数:

```
// 1. 默认构造: 创建空的 unique_ptr
unique_ptr<int> p1; // 空指针, 不指向任何内存
if (p1 == nullptr) {
    cout << "p1 是空的 unique_ptr" << endl;
}
```

-----下面的为执行结果-----

p1 是空的 unique_ptr

2. 指针构造函数:

```
// 2. 指针构造: 用裸指针初始化, 接管内存所有权
unique_ptr<int> p2(new int(10)); // p2 独占指向值为10的int内存
cout << "p2 指向的值: " << *p2 << endl; // 解引用获取值
```

-----下面的为执行结果-----

p2 指向的值: 10

3. 移动构造函数:

```
// 3. 移动构造: 转移所有权 (p2 -> p3)
unique_ptr<int> p3(move(p2)); // 把p2的所有权转给p3, p2变空
if (p2 == nullptr) {
    cout << "p2 转移后为空" << endl;
}
cout << "p3 接管的值: " << *p3 << endl; // p3 现在指向10
```

-----下面的为执行结果-----

p2 转移后为空

p3 接管的值: 10

4. reset 方法初始化

reset 的函数原型如下:

```
void reset( pointer ptr = pointer() ) noexcept;
```

使用 **reset** 方法可以让 **unique_ptr** 解除对原始内存的管理，也可以用来初始化一个独占的智能指针。

实例代码如下：

```
#include <iostream>
#include <memory>
using namespace std;

int main() {
    // 先默认构造一个空的 unique_ptr
    unique_ptr<int> p;
    // 用 reset() 初始化：给空指针赋值新资源
    p.reset(new int(20));
    cout << "p 通过 reset 初始化后的值: " << *p << endl;
    // reset(nullptr) 可释放资源并置空
    p.reset();
    if (p == nullptr) {
        cout << "p 置空后为空" << endl;
    }
    // reset() 也可重置已有资源（释放旧的，指向新的）
    p.reset(new int(30));
    cout << "p 重置后的值: " << *p << endl;
    return 0;
}

-----下面的为执行结果-----
p 通过 reset 初始化后的值: 20
p 置空后为空
p 重置后的值: 30
```

- `p.reset()`：解除对原始内存的管理
- `p.reset(new int(30))`：重新指定智能指针管理的原始内存

2.3 获取智能指针管理的原始地址

如果想要获取独占智能指针管理的原始地址，可以调用 `get()` 方法，函数原型如下：

```
pointer get() const noexcept;
```

实例代码如下：

```
int main()
{
    unique_ptr<int> ptr1(new int(10));
    unique_ptr<int> ptr2 = move(ptr1);

    ptr2.reset(new int(250));
    cout << *ptr2.get() << endl; // 得到内存地址中存储的实际数值 250
    return 0;
}
```

2.4 删除器

删除器是一个可调用对象（函数、lambda、函数对象等），`unique_ptr` 会在自身析构（或调用 `reset()`）时，自动调用这个删除器来释放所管理的资源，而非默认的 `delete`。主要的目的是突破默认 `delete` 的限制，适配不同类型的资源释放逻辑（比如动态数组需要 `delete[]`、文件句柄需要 `fclose()` 等）。

1. 默认删除器

`unique_ptr` 有默认的删除器：

- 对于普通指针（`unique_ptr`）：默认调用 `delete ptr`
- 对于数组指针（`unique_ptr<T[]>`）：默认调用 `delete[] ptr`

示例代码如下：

```
#include <memory>
#include <iostream>
using namespace std;

int main() {
    // 普通指针：默认 delete
    unique_ptr<int> p1(new int(10));
    // 数组指针：默认 delete[]
    unique_ptr<int[]> p2(new int[3]{1,2,3});
    return 0; // 析构时自动调用对应删除逻辑
}
```

2. 自定义删除器的使用场景:

默认删除器只能处理 `new/delete` 管理的内存，以下场景需要自定义删除器：

- 释放系统资源（如文件句柄、网络套接字、内存映射等）；
- 释放第三方库分配的内存（如用 `malloc` 分配、需 `free` 释放）；
- 释放时需要额外逻辑（如打印日志、统计释放次数）

3. 函数作为删除器的可调用对象

示例代码如下：

```
#include <memory>
#include <iostream>
using namespace std;

// 自定义删除器函数：释放 int 数组
void arrayDeleter(int* ptr) {
    if (ptr != nullptr) {
        cout << "释放 int 数组，首元素: " << ptr[0] << endl;
        delete[] ptr; // 数组必须用 delete[]
    }
}

int main() {
    // 声明时指定删除器类型（函数指针类型）
    unique_ptr<int, void(*)(int*)> p(new int[3]{10,20,30}, arrayDeleter);
    //用 get() 获取原始指针后访问
    cout << "p 管理的数组首元素: " << p.get()[0] << endl;
    return 0; // 析构时自动调用 arrayDeleter
}

-----下面的为执行结果-----
p 管理的数组首元素: 10
释放 int 数组，首元素: 10
```

4. Lambda 表达式作为删除器的可调用对象

```
#include <memory>
#include <iostream>
using namespace std;

int main() {
    // Lambda 作为删除器（释放 malloc 分配的内存）
```

```

auto freeDeleter = [](int* ptr) {
    cout << "释放 malloc 分配的内存, 值: " << *ptr << endl;
    free(ptr); // 替代 delete
};

// Lambda 类型无法直接写, 用 auto 推导 + decltype
unique_ptr<int, decltype(freeDeleter)> p((int*)malloc(sizeof(int)),
freeDeleter);
*p = 100; // 赋值
cout << "p 管理的 malloc 内存值: " << *p << endl;
return 0; // 析构时调用 Lambda 中的 free
}

-----下面的为执行结果-----
p 管理的 malloc 内存值: 100
释放 malloc 分配的内存, 值: 100

```

三、std::shared_ptr---共享的智能指针

3.1 定义与用法

共享智能指针是指多个智能指针可以同时管理同一块有效的内存，共享智能指针 **shared_ptr** 是一个模板类，如果要进行初始化有三种方式：通过构造函数、**std::make_shared** 辅助函数以及 **reset** 方法。共享智能指针对象初始化完毕之后就指向了要管理的那块堆内存。通过引用计数机制，管理资源的生命周期。

主要特性：

- 共享所有权：多个 **shared_ptr** 可以指向同一个对象。
- 引用计数：跟踪有多少 **shared_ptr** 实例指向同一对象。
- 自动释放：当引用计数为0时，自动释放资源。

如果想要查看当前有多少个智能指针同时管理着这块内存可以使用共享智能指针提供的一个成员函数 **use_count**，函数原型如下：

```

// 管理当前对象的 shared_ptr 实例数量，或若无被管理对象则为 0。
long use_count() const noexcept;

```


3.2 初始化

`shared_ptr` 提供多种构造函数和赋值操作，以支持不同的使用场景。

- 默认构造函数：创建一个空的 `shared_ptr`。
- 指针构造函数：接受一个裸指针，拥有其所有权。
- 拷贝构造函数：增加引用计数，共享对象所有权。
- 移动构造函数：转移所有权，源 `shared_ptr` 变为空。
- 通过 `std::make_shared` 初始化
- 使用 `reset` 方法初始化

1. 默认构造函数：创建一个空 `shared_ptr`

```
// ===== 1. 默认构造函数：创建空的 shared_ptr =====
shared_ptr<int> p1; // 空指针，不指向任何资源
cout << "1. 默认构造 p1: 引用计数 = " << p1.use_count()
    << ", 是否为空 = " << (p1 == nullptr) << endl;
-----下面的为执行结果-----
1. 默认构造 p1: 引用计数 = 0, 是否为空 = 1
```

2. 指针构造函数：接受一个裸指针，拥有其所有权

如果智能指针被初始化了一块有效内存，那么这块内存的引用计数+1，如果智能指针没有被初始化或者被初始化为 `nullptr` 空指针，引用计数不会+1。另外，不要使用一个原始指针初始化多个 `shared_ptr`。

```
// ===== 2. 指针构造函数：接受裸指针，接管所有权 =====
shared_ptr<int> p2(new int(10)); // 接管值为10的int内存
cout << "2. 指针构造 p2: 引用计数 = " << p2.use_count()
    << ", 指向的值 = " << *p2 << endl;
-----下面的为执行结果-----
2. 指针构造 p2: 引用计数 = 1, 指向的值 = 10
```

3. 拷贝构造函数：增加引用计数，共享对象所有权

如果使用拷贝的方式初始化共享智能指针对象，这两个对象会同时管理同一块堆内存，堆内存对应的引用计数也会增加；

```
// ===== 3. 拷贝构造函数：共享所有权，引用计数+1 =====
shared_ptr<int> p3(p2); // 从p2拷贝，共享资源
cout << "3. 拷贝构造 p3(p2): p2计数 = " << p2.use_count()
    << ", p3计数 = " << p3.use_count()
    << ", 指向的值 = " << *p3 << endl;

-----下面的为执行结果-----
3. 拷贝构造 p3(p2): p2计数 = 2, p3计数 = 2, 指向的值 = 10
```

4. 移动构造函数：转移所有权，源 shared_ptr 变为空

如果使用移动的方式初始智能指针对象，只是转让了内存的所有权，管理内存的对象并不会增加，因此内存的引用计数不会变化。

```
// ===== 4. 移动构造函数：转移所有权，源指针变空 =====
shared_ptr<int> p4(move(p2)); // 从p2移动，p2变为空
cout << "4. 移动构造 p4(move(p2)): p2是否为空 = " << (p2 == nullptr)
    << ", p4计数 = " << p4.use_count()
    << ", p3计数 = " << p3.use_count() << endl;

-----下面的为执行结果-----
4. 移动构造 p4(move(p2)): p2是否为空 = 1, p4计数 = 2, p3计数 = 2
```

5. 通过std::make_shared初始化

std::make_shared是一个模板函数，用于一次性分配并初始化 shared_ptr 管理的对象内存，以及 shared_ptr 内部的控制块（存储引用计数等信息），最终返回一个 shared_ptr 实例。

核心优势：

- 内存效率更高：普通 shared_ptr(new T)会分配两次内存（一次给对象，一次给控制块）；make_shared只分配一次连续内存（对象 + 控制块），减少内存碎片
- 异常安全：避免裸指针构造时可能的内存泄漏
- 代码更简洁：无需手动写 new，减少出错概率

使用std::make_shared()模板函数可以完成内存地址的创建，并将最终得到的内存地址传递给共享智能指针对象管理。如果申请的内存是普通类型，通过函数的（）可完成地址的初始化，如果要创建一个类对象，函数的（）内部需要指定构造对象需要的参数，也就是类构造函数的参数。

示例代码:

```
#include <iostream>
#include <string>
#include <memory>
using namespace std;

class Test
{
public:
    Test()
    {
        cout << "construct Test..." << endl;
    }
    Test(int x)
    {
        cout << "construct Test, x = " << x << endl;
    }
    Test(string str)
    {
        cout << "construct Test, str = " << str << endl;
    }
    ~Test()
    {
        cout << "destruct Test ..." << endl;
    }
};

int main()
{
    // 使用智能指针管理一块 int 型的堆内存, 内部引用计数为 1
    shared_ptr<int> ptr1 = make_shared<int>(520);
    cout << "ptr1管理的内存引用计数: " << ptr1.use_count() << endl;

    shared_ptr<Test> ptr2 = make_shared<Test>();
    cout << "ptr2管理的内存引用计数: " << ptr2.use_count() << endl;

    shared_ptr<Test> ptr3 = make_shared<Test>(520);
    cout << "ptr3管理的内存引用计数: " << ptr3.use_count() << endl;

    shared_ptr<Test> ptr4 = make_shared<Test>("我要学会c++!!!");
```

```

    cout << "ptr4管理的内存引用计数: " << ptr4.use_count() << endl;
    return 0;
}

```

-----下面的为执行结果-----

```

ptr1管理的内存引用计数: 1
construct Test...
ptr2管理的内存引用计数: 1
construct Test, x = 520
ptr3管理的内存引用计数: 1
construct Test, str = 我要学会c++!!!
ptr4管理的内存引用计数: 1
destruct Test ...
destruct Test ...
destruct Test ...

```

6. 通过reset方法初始化

对于一个未初始化的共享智能指针，可以通过reset方法来初始化，当智能指针中有值的时候，调用reset会使引用计数减1。

示例代码：

```

#include <iostream>
#include <string>
#include <memory>
using namespace std;

int main()
{
    // 使用智能指针管理一块 int 型的堆内存, 内部引用计数为 1
    shared_ptr<int> ptr1 = make_shared<int>(520);
    shared_ptr<int> ptr2 = ptr1;
    shared_ptr<int> ptr3 = ptr1;
    shared_ptr<int> ptr4 = ptr1;
    cout << "ptr1管理的内存引用计数: " << ptr1.use_count() << endl;
    cout << "ptr2管理的内存引用计数: " << ptr2.use_count() << endl;
    cout << "ptr3管理的内存引用计数: " << ptr3.use_count() << endl;
    cout << "ptr4管理的内存引用计数: " << ptr4.use_count() << endl;

    ptr4.reset();
}

```

```

cout << "ptr1管理的内存引用计数: " << ptr1.use_count() << endl;
cout << "ptr2管理的内存引用计数: " << ptr2.use_count() << endl;
cout << "ptr3管理的内存引用计数: " << ptr3.use_count() << endl;
cout << "ptr4管理的内存引用计数: " << ptr4.use_count() << endl;

shared_ptr<int> ptr5;
ptr5.reset(new int(250));
cout << "ptr5管理的内存引用计数: " << ptr5.use_count() << endl;

return 0;
}

```

-----下面的为执行结果-----

```

ptr1管理的内存引用计数: 4
ptr2管理的内存引用计数: 4
ptr3管理的内存引用计数: 4
ptr4管理的内存引用计数: 4
ptr1管理的内存引用计数: 3
ptr2管理的内存引用计数: 3
ptr3管理的内存引用计数: 3
ptr4管理的内存引用计数: 0
ptr5管理的内存引用计数: 1

```

3.3 获取智能指针管理的原始地址

通过智能指针可以管理一个普通变量或者对象的地址，此时原始地址就不可见了。当我们想要修改变量或者对象中的值的时候，就需要从智能指针对象中先取出数据的原始内存的地址再操作，解决方案是调用共享智能指针类提供的get方法。函数原型如下：

```
T* get() const noexcept;
```

示例代码：

```

#include <iostream>
#include <string>
#include <cstring>
#include <memory>
using namespace std;

```

```

int main()
{
    int len = 128;
    shared_ptr<char> ptr(new char[len]);
    // 得到指针的原始地址
    char* add = ptr.get();
    memset(add, 0, len);
    strcpy(add, "我要成为最懂c++的男人!!!");
    cout << "string: " << add << endl;

    shared_ptr<int> p(new int);
    *p = 100;
    cout << *p.get() << " " << *p << endl;

    return 0;
}

```

-----下面的为执行结果-----

```

string: 我要成为最懂c++的男人!!!
100 100

```

3.4 删除器

当智能指针管理的内存对应的引用计数变为0的时候，这块内存就会被智能指针析构掉了。另外，我们在初始化智能指针的时候也可以自己指定删除动作，这个删除操作对应的函数被称之为删除器，这个删除器函数本质是一个回调函数，我们只需要进行实现，其调用是由智能指针完成的

示例代码：

```

#include <iostream>
#include <memory>
using namespace std;

// 自定义删除器函数，释放int型内存
void deleteIntPtr(int* p)
{
    delete p;
    cout << "int 型内存被释放了...";
}

```

```
int main()
{
    shared_ptr<int> ptr(new int(250), deleteIntPtr);
    return 0;
}
```

-----下面的为执行结果-----

int 型内存被释放了...

删除器函数也可以是lambda表达式，因此代码也可以写成下面这样：

```
int main()
{
    shared_ptr<int> ptr(new int(250), [](int* p) {delete p; });
    return 0;
}
```

在上面的代码中，lambda表达式的参数就是智能指针管理的内存的地址，有了这个地址之后函数体内部就可以完成删除操作了。

在C++11中使用shared_ptr管理动态数组时，需要指定删除器，因为std::shared_ptr的默认删除器不支持数组对象，具体的处理代码如下：

```
int main()
{
    shared_ptr<int> ptr(new int[10], [](int* p) {delete[]p; });
    return 0;
}
```

四、std::weak_ptr---弱引用的智能指针

4.1 定义与用法

弱引用智能指针 `std::weak_ptr` 可以看做是 `std::shared_ptr` 的助手，它不管理 `std::shared_ptr` 内部的指针。`std::weak_ptr` 没有重载操作符 `*` 和 `->`，因为它不共享指针，不能操作资源，所以它的构造不会增加引用计数，析构也不会减少引用计数，它的主要作用就是作为一个旁观者监视 `std::shared_ptr` 中管理的资源是否存在，用于解决 `std::shared_ptr` 之间的循环引用问题。

主要特性：

- 非拥有所有权：不增加引用次数
- 可从 `std::shared_ptr` 生成：通过 `std::weak_ptr` 可以访问 `shared_ptr` 管理的对象。
- 避免循环引用：适用于双向关联或观察者模式

4.2 初始化

初始化的全部示例代码：

```
#include <iostream>
#include <memory>
using namespace std;

int main()
{
    shared_ptr<int> sp(new int);

    weak_ptr<int> wp1;
    weak_ptr<int> wp2(wp1);
    weak_ptr<int> wp3(sp);
    weak_ptr<int> wp4;
    wp4 = sp;
    weak_ptr<int> wp5;
    wp5 = wp3;

    cout << "use_count: " << endl;
    cout << "wp1: " << wp1.use_count() << endl;
    cout << "wp2: " << wp2.use_count() << endl;
    cout << "wp3: " << wp3.use_count() << endl;
    cout << "wp4: " << wp4.use_count() << endl;
    cout << "wp5: " << wp5.use_count() << endl;
```



```
    return 0;  
}
```

1. 默认构造

示例代码:

```
weak_ptr<int> wp1;
```

`weak_ptr wp1`:构造了一个空`weak_ptr`对象

2. 拷贝构造

示例代码:

```
weak_ptr<int> wp2(wp1);  
-----第二种-----  
weak_ptr<int> wp5;  
wp5 = wp3;
```

`weak_ptr wp2(wp1)`:通过一个空`weak_ptr`对象构造了另一个空`weak_ptr`对象

`wp5 = wp3`: 通过一个`weak_ptr`对象构造了一个可用的`weak_ptr`实例对象

3. 通过`shared_ptr`对象构造

示例代码:

```
shared_ptr<int> sp(new int);  
weak_ptr<int> wp3(sp);  
-----第二种-----  
weak_ptr<int> wp4;  
wp4 = sp;
```

`weak_ptr wp3(sp)`:通过一个`shared_ptr`对象构造了一个可用的`weak_ptr`实例对象

wp4 = sp;通过一个shared_ptr对象构造了一个可用的weak_ptr实例对象（这是一个隐式类型转换）

4. 打印所有的引用值

```
use_count:  
wp1: 0  
wp2: 0  
wp3: 1  
wp4: 1  
wp5: 1
```

通过打印的结果可以知道，虽然弱引用智能指针wp3、wp4、wp5监测的资源是同一个，但是它的引用计数并没有发生任何的变化，也进一步证明了weak_ptr只是监测资源，并不管理资源。

4.3 其他常用方法

1. expired()方法

通过调用std::weak_ptr类提供的expired()方法来判断观测的资源是否已经被释放，函数原型如下：

```
// 返回true表示资源已经被释放, 返回false表示资源没有被释放  
bool expired() const noexcept;
```

示例代码：

```
#include <iostream>  
#include <memory>  
using namespace std;  
  
int main()  
{  
    shared_ptr<int> shared(new int(10));  
    weak_ptr<int> weak(shared);  
    cout << "1. weak " << (weak.expired() ? "is" : "is not") << " expired" << endl;
```

```

shared.reset();
cout << "2. weak " << (weak.expired() ? "is" : "is not") << " expired" << endl;

return 0;
}

```

-----下面的为执行结果-----

1. weak is not expired
2. weak is expired

weak_ptr监测的就是shared_ptr管理的资源，当共享智能指针调用shared.reset();之后管理的资源被释放，因此weak.expired()函数的结果返回true，表示监测的资源已经不存在了。

2. lock()方法

通过调用std::weak_ptr类提供的lock()方法来获取管理所监测资源的shared_ptr对象，函数原型如下：

```
shared_ptr<element_type> lock() const noexcept;
```

示例代码：

```

#include <iostream>
#include <memory>
using namespace std;

int main()
{
    shared_ptr<int> sp1, sp2;
    weak_ptr<int> wp;

    sp1 = std::make_shared<int>(520);
    wp = sp1;
    sp2 = wp.lock();
    cout << "use_count: " << wp.use_count() << endl;

    sp1.reset();
    cout << "use_count: " << wp.use_count() << endl;
}

```

```

    sp1 = wp.lock();
    cout << "use_count: " << wp.use_count() << endl;

    cout << "*sp1: " << *sp1 << endl;
    cout << "*sp2: " << *sp2 << endl;

    return 0;
}

```

-----下面的为执行结果-----

```

use_count: 2
use_count: 1
use_count: 2
*sp1: 520
*sp2: 520

```

- `sp2 = wp.lock();`通过调用`lock()`方法得到一个用于管理`weak_ptr`对象所监测的资源的共享智能指针对象，使用这个对象初始化`sp2`，此时所监测资源的引用计数为2
- `sp1.reset();`共享智能指针`sp1`被重置，`weak_ptr`对象所监测的资源的引用计数减1
- `sp1 = wp.lock();``sp1`重新被初始化，并且管理的还是`weak_ptr`对象所监测的资源，因此引用计数加1
- 共享智能指针对象`sp1`和`sp2`管理的是同一块内存，因此最终打印的内存中的结果是相同的，都是520

3. `reset()`方法

通过调用`std::weak_ptr`类提供的`reset()`方法来清空对象，使其不监测任何资源，函数原型如下：

```
void reset() noexcept;
```

示例代码：

```

#include <iostream>
#include <memory>
using namespace std;

int main()
{
    shared_ptr<int> sp(new int(10));
}

```

```

weak_ptr<int> wp(sp);
cout << "1. wp " << (wp.expired() ? "is" : "is not") << " expired" << endl;

wp.reset();
cout << "2. wp " << (wp.expired() ? "is" : "is not") << " expired" << endl;

return 0;
}
-----下面的为执行结果-----
1. wp is not expired
2. wp is expired

```

weak_ptr对象sp被重置之后wp.reset();变成了空对象，不再监测任何资源，因此wp.expired()返回true

4.4 解决循环引用的问题

1. 什么是循环引用问题

shared_ptr 的核心是强引用计数：计数为 0 时释放对象。但如果两个（或多个）对象通过 shared_ptr 互相引用，会形成“闭环”，导致它们的强引用计数永远无法归 0，最终对象无法析构，造成内存泄漏。

示例代码：

```

#include <iostream>
#include <memory>
using namespace std;

// 两个类互相引用
class B; // 前向声明
class A {
public:
    shared_ptr<B> b_ptr; // A 持有 B 的 shared_ptr
    ~A() { cout << "A 析构" << endl; } // 析构函数验证是否释放
};

class B {
public:
    shared_ptr<A> a_ptr; // B 持有 A 的 shared_ptr
    ~B() { cout << "B 析构" << endl; }
}

```

```
};
int main() {
    // 创建两个对象，互相引用
    shared_ptr<A> a = make_shared<A>();
    shared_ptr<B> b = make_shared<B>();
    cout << "a 的引用计数: " << a.use_count() << endl;
    cout << "b 的引用计数: " << b.use_count() << endl;
    a->b_ptr = b; // A 引用 B
    b->a_ptr = a; // B 引用 A
    // 查看引用计数
    cout << "a 的引用计数: " << a.use_count() << endl;
    cout << "b 的引用计数: " << b.use_count() << endl;
    return 0;
}
```

-----下面的为执行结果-----

```
a 的引用计数: 1
b 的引用计数: 1
a 的引用计数: 2
b 的引用计数: 2
```

测试程序中，共享智能指针a、b对A、B实例对象的引用计数变为2，在共享智能指针离开作用域之后引用计数只能减为1，这种情况下不会去删除智能指针管理的内存，导致类A、B的实例对象不能被析构，最终造成内存泄露。

2. weak_ptr 如何解决循环引用

weak_ptr 是弱引用，核心特性：

- + 指向 shared_ptr 管理的对象，但**不增加引用计数**；
- + 只是“观察”对象，不会阻止对象被释放
- + 要访问对象时，需通过 lock() 方法生成临时的 shared_ptr（此时才会临时增加计数，访问完后计数减少

只需把其中一个 **shared_ptr** 换成 **weak_ptr** 即可（比如把 B 中的 a_ptr 改为 weak_ptr）

示例代码：

```
#include <iostream>
#include <memory>
```

```

using namespace std;

-
class B;
class A {
public:
    shared_ptr<B> b_ptr;
    ~A() { cout << "A 析构" << endl; }
};

-
class B {
public:
    weak_ptr<A> a_ptr; // 关键: 换成 weak_ptr, 不再增加引用计数
    ~B() { cout << "B 析构" << endl; }
};

-
int main() {
    shared_ptr<A> a = make_shared<A>();
    shared_ptr<B> b = make_shared<B>();

    a->b_ptr = b; // A 引用 B (强引用, b 的计数变为 2)
    b->a_ptr = a; // B 引用 A (弱引用, a 的计数仍为 1)
    // 查看计数
    cout << "a 的引用计数: " << a.use_count() << endl;
    cout << "b 的引用计数: " << b.use_count() << endl;
    // 通过 weak_ptr 访问对象: 用 lock() 生成 shared_ptr
    if (shared_ptr<A> temp = b->a_ptr.lock()) {
        cout << "成功访问 A 对象" << endl;
    }
    return 0;
}

```

-----下面的为执行结果-----

```

a 的引用计数: 1
b 的引用计数: 2
成功访问 A 对象
A 析构
B 析构

```